

Space to Steer, Text to Command: Comparing Infinite-Canvas and Terminal Interfaces for Multi-Agent Supervision

Anonymous Authors

Anonymous Institution

Anonymous City, Anonymous Country

anonymous@example.com

ABSTRACT

Multi-agent AI systems are increasingly used for research, coding, and other long-running knowledge-work tasks, yet the interfaces that expose these systems to users are diverging into two distinct families. One family centers on terminal-like command streams that foreground direct invocation, linear logs, and low-friction repair. The other centers on spatial workspaces that externalize agent roles, dependencies, and intermediate artifacts on a shared canvas. Existing HCI work has introduced compelling tools in both families, but the field still lacks a matched comparison of how these representational choices shape human supervision when the underlying agent capability is held constant. We address this gap by contributing a local-first research package for comparing infinite-canvas and terminal interfaces in multi-agent supervision. The package includes a matched two-condition prototype, a counterbalanced mixed-methods study protocol, a measurement and logging plan, and a synthetic dry run that validates the analysis and manuscript pipeline. In the synthetic dry run, a planned within-subject dataset of 24 participants was instantiated as 144 task rows, of which two participants were intentionally excluded to validate preprocessing. Descriptive patterns favored the canvas condition for monitoring, plan recall, and workload reduction, while the terminal condition preserved an advantage for direct recovery speed. We do not treat these dry-run numbers as empirical findings. Instead, we use them to show how a representation-centered comparison can structure future CHI research on agent interfaces and to argue that hybrid tools should combine spatial overview with command-line precision.

ACM Reference Format:

Anonymous Authors. 2026. Space to Steer, Text to Command: Comparing Infinite-Canvas and Terminal Interfaces for Multi-Agent Supervision. In *Proceedings of ACM Conference (Conference'17)*. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 INTRODUCTION

As large language models move from single-shot question answering to extended task execution, the human interface to those models is becoming a core research problem rather than a secondary implementation detail. The emerging shift is not only from single assistants to multiple agents, but also from short exchanges to coordinated workflows that include planning, branching, monitoring, repair, and synthesis. In these settings, the user does not merely “ask the model” for an answer. Instead, the user supervises several active processes, decides when one subtask should block another,

inspects intermediate outputs, and attempts to recover when the system drifts. That supervisory work is representationally demanding. It depends on what the interface makes visible, what remains hidden in logs or memory, and how expensive it is to intervene at the right moment.

Two interface families have emerged as dominant responses to this problem. The first is terminal-like. It treats agentic work as a command-and-log stream in which users type instructions, inspect chronological traces, and redirect execution through textual commands. Recent terminal-oriented arguments claim that this design remains attractive because it preserves representational compatibility with underlying tools, exposes precise action history, and keeps repair operations compact [3]. The second family is spatial. It represents workflows as cards, nodes, boards, or notebook-like regions in which users can place subtasks, inspect dependencies, and hold several states in view at once. Prior systems in this family have explored visual chaining, prompt evaluation, notebook restructuring, no-code agent assembly, and overview-oriented debugging [2, 4, 5, 8–10, 12].

The coexistence of these two families makes interface choice more than a stylistic preference. It raises a deeper HCI question about how representation redistributes cognitive work. A terminal may offer explicit chronology and fast command entry, yet still require the user to reconstruct high-level structure from a dense stream. A spatial canvas may offload memory into visible layout, yet still introduce navigation or manipulation overhead when urgent interventions are needed. In other words, the comparison is not “visual versus textual” in the abstract. It is a question of how users plan, monitor, and repair multi-agent workflows when the same backend capability is rendered through different representational assumptions.

This question matters because current multi-agent systems are particularly vulnerable to hidden state. Agents may branch, block, drift, or overwrite earlier assumptions without a user noticing immediately. Systems such as AutoGen Studio, AGDebugger, Magentic-UI, and Cocoa already suggest that oversight, steering, and co-planning deserve dedicated interaction support [4–6, 11]. At the same time, visual prompt engineering tools such as PromptChainer, ChainForge, and EvalLM show that once a workflow becomes multi-step and comparative, users often benefit from representations that externalize structure rather than bury it in serialized history [2, 8, 12]. Spatial sensemaking research further suggests that layout is not cosmetic: it can serve as external memory and analytic scaffolding [1]. Yet none of these threads, on their own, answer the concrete comparison now facing practitioners and tool builders: when users supervise several agents on matched tasks, what changes if they do so on an infinite canvas rather than in a terminal?

This paper takes that gap as its starting point. Rather than claim a completed human-subject experiment, we present a study-ready research package designed to make the comparison rigorous. The package includes a matched prototype that exposes the same scripted multi-agent backend through two interfaces, a mixed-methods within-subject study design, task scenarios that isolate delegation, monitoring, and recovery, and a synthetic dry run that validates the planned analysis and writing pipeline. The dry run is intentionally non-empirical. Its purpose is not to prove that one interface is better. Its purpose is to ensure that the evaluation structure, measures, outputs, and argumentation are coherent enough to support a real study later.

We make four contributions. First, we frame multi-agent interface design as a representational coordination problem, arguing that interface choice redistributes memory load, oversight timing, and repair effort. Second, we contribute a matched comparison prototype that makes the infinite-canvas versus terminal contrast concrete without changing backend functionality. Third, we contribute a study protocol and analysis plan that operationalize the comparison across task success, workload, plan recall, and qualitative strategy reports. Fourth, we contribute a synthetic dry run that demonstrates how such a study can be instrumented, summarized, and communicated in a CHI-style manuscript without overclaiming evidence.

The rest of the paper positions this contribution in prior HCI work on spatial sensemaking, visual AI tooling, and agent oversight. We then describe the prototype and study design, report the dry-run outputs as descriptive validation, and discuss what the comparison implies for future hybrid agent interfaces. Our central argument is simple: multi-agent supervision should be studied not only in terms of model capability, but also in terms of how representation shapes human control.

2 RELATED WORK

Our work sits at the intersection of four literatures: spatial sensemaking and external cognition, visual prompt and workflow tools, interfaces for human oversight of multi-agent systems, and emerging terminal-oriented theories of agent interaction. None of these literatures alone resolves our comparison, but together they clarify why the comparison is overdue.

The first thread comes from HCI and visualization research on space, memory, and analytic work. Andrews et al. showed that large, high-resolution displays support sensemaking by giving users room to externalize intermediate structure, maintain context, and organize evidence spatially [1]. The core implication is that layout can function as cognitive infrastructure. People use space not only to display information, but also to remember what has been done, identify gaps, and keep partially resolved ideas in view. More recently, InterLink showed that even in computational notebooks, alternative layouts can improve interpretation accuracy for complex materials by changing how text, code, and output relate visually [9]. These works do not involve AI agents, but they are directly relevant because multi-agent supervision is also a sensemaking problem. Users must relate steps, branches, artifacts, and failures across time. A spatial canvas plausibly helps because it offers persistent

external structure; a terminal plausibly constrains because it serializes structure into history.

The second thread concerns visual tools for prompting, evaluation, and workflow composition with language models. PromptChainer introduced a visual programming approach for chaining LLM prompts, arguing that multi-step tasks need transparency and compositional control [12]. ChainForge extended this direction by supporting prompt engineering and hypothesis testing across models through a visual toolkit designed around iterative exploration and comparison [2]. EvalLM focused on interactive prompt evaluation using user-defined criteria, showing that structured comparison can help users inspect more outputs and revise prompts more deliberately [8]. PromptAid likewise explored visual analytics for prompt exploration and iteration [10], while LLM Comparator provided a visual analysis environment for side-by-side evaluation of model behavior at larger scale [7]. Across these systems, the repeated lesson is that once work becomes multi-output, multi-criterion, or iterative, visual structure helps users compare states and reason about alternatives. However, these systems are oriented toward prompt design, evaluation, or model comparison rather than real-time supervision of multiple semi-autonomous agents. They demonstrate the promise of visual externalization, but they do not compare it against terminal-style control under matched task conditions.

The third thread addresses human oversight of multi-agent systems more directly. AutoGen Studio lowered the barrier to assembling and debugging multi-agent systems through a no-code developer environment [4]. AGDebugger emphasized that long multi-agent conversations are difficult to inspect without overview mechanisms, reset operations, and ways to steer execution after the fact [5]. Magentic-UI pushed further toward human-in-the-loop agentic systems by combining co-planning, multitasking, and action guards into a unified interface model [11]. Cocoa, a recent CHI paper, proposed co-planning and co-execution structures that preserve user steerability in long-running tasks [6]. These works strongly support the idea that agent oversight is an interface problem, not only a backend one. They show that users need to understand state, interrupt behavior, and redirect plans. Yet most of these systems study a specific interface design in isolation. They help answer “what should an agent interface provide?” but not “how do different representational families shift the burden of providing it?”

The fourth thread comes from the resurgence of terminal-oriented thinking in AI tooling. As more coding and orchestration tools are distributed through command-first environments, the terminal is being defended not as a legacy artifact but as a deliberately strong interaction substrate. De Masi argues that terminal interfaces remain attractive for human-AI agent collaboration because they expose procedure directly, minimize friction between intent and action, and align closely with the execution environment [3]. This argument is important because it challenges any simple assumption that newer spatial interfaces necessarily dominate older textual ones. In practice, many expert users continue to prefer textual tools for recovery, composition, and scripting precisely because those tools make low-level operations concise. However, this literature is still mostly normative or conceptual. It tells us why terminal tools remain compelling, but not how their strengths interact with the specific coordination demands of multi-agent supervision.

Taken together, these literatures suggest a missing middle. Spatial work helps with overview, external memory, and branch comparison. Terminal work helps with procedural transparency, precision, and compact action history. Visual AI tools and agent-debugging systems show that users need structured representations once tasks become multi-step. Agentic workflow systems show that oversight and intervention are now ordinary rather than exceptional. What remains insufficiently studied is the representational tradeoff itself: when identical multi-agent tasks are presented through a spatial infinite canvas versus a terminal log-and-command stream, how do users' strategies, breakdowns, and perceived control change?

Our work addresses that gap by designing the comparison into the artifact. We do not propose yet another visual tool or yet another terminal. Instead, we hold the backend constant and make representation the independent variable. That framing lets us connect the spatial cognition literature to the agent interface literature in a way that more directly serves CHI's interest in how interface form shapes human work.

3 METHOD

This paper reports a study-ready research package rather than a completed human-subject experiment. The method therefore has two layers. The first layer is the intended study design: a counter-balanced within-subject comparison between infinite-canvas and terminal interfaces for supervising the same multi-agent workflow. The second layer is a synthetic dry run that instantiates the planned dataset and analysis outputs so that the instrumentation, summaries, and manuscript logic can be validated locally before any recruitment.

3.1 Research objective

Our objective is to compare two representational modalities for multi-agent supervision while minimizing confounds from backend differences. Many real systems combine interface changes with capability changes, which makes it difficult to know whether observed benefits arise from better models, different task support, or the representation itself. We therefore designed a matched comparison in which the same underlying workflow, branch points, and intervention opportunities are presented in two ways. In the *terminal* condition, users act through commands and inspect chronological logs. In the *canvas* condition, users act through spatial cards, visible dependencies, and board-level state. Both conditions expose the same subtasks, agent roles, and failure cues.

3.2 Prototype design

The prototype is a high-fidelity local-first mockup rather than a production agent runtime. This choice is deliberate. For the comparison we needed stable, repeatable task conditions with identical branch structure, not the variability of live model execution. The prototype therefore scripts three scenarios: delegation, monitoring, and recovery. In the delegation scenario, participants decompose a research request across scout, synthesis, and writing agents while noticing a missing evaluation checkpoint. In the monitoring scenario, participants inspect overlap, stalled state, and missing terminal-side evidence across several active branches. In the

recovery scenario, participants trace a stale writer branch back to an outdated synthesis checkpoint and restore the workflow.

In the canvas condition, the interface displays agent roles and artifacts as spatial cards on a large board. Dependencies are visible as edges. State badges indicate whether a node is running, blocked, conflicting, or ready to steer. The board is designed to make absence visible: if an expected evaluation step is missing, the gap appears structurally rather than only as a buried warning. In the terminal condition, the interface displays a linear command-and-log stream. Users can inspect state, pause agents, rebind inputs, and restart execution through compact textual commands. This condition is designed to preserve the strengths that terminal advocates emphasize: directness, precise repair, and chronological traceability [3].

The prototype records the same conceptual events in both conditions: task start, intervention, target agent, failure cue, and task completion. The mockup is stored locally under the project prototype directory and is meant to support walkthroughs, pilot sessions, and interface screenshots without requiring a public server. The resulting comparison is not about one condition having more features. It is about whether identical capabilities feel and behave differently when the user must supervise them through spatial overview or serialized command history.

3.3 Planned participants and procedure

The intended real study recruits 24 to 30 AI-literate knowledge workers who use LLMs at least weekly for work or study. We deliberately target users who are comfortable with extended, tool-mediated work rather than total novices, because the comparison is most meaningful when participants can engage in genuine supervisory behavior rather than basic interface discovery. At the same time, we plan to balance prior familiarity with terminals and visual workflow tools so that the dataset captures differences in representational fit rather than only prior expertise.

The planned procedure is counterbalanced within-subjects. After consent and a short background questionnaire, each participant receives a tutorial for the first interface condition, completes three scenario tasks, fills out a post-condition questionnaire, and then repeats the process with the second condition. The study ends with a comparative interview. The entire session is designed to last about 70 minutes. The within-subject design allows us to ask not only which condition produces better scores on specific tasks, but also whether users carry different strategies, frustrations, or intervention patterns from one modality to the other.

The three tasks were selected to represent distinct supervisory demands. Delegation asks whether the interface helps users externalize structure before execution starts. Monitoring asks whether the interface helps users maintain awareness across several partially completed branches. Recovery asks whether the interface helps users locate the source of a breakdown and repair it efficiently. This decomposition matters because we do not expect a single interface to dominate all forms of work. A terminal may excel at direct repair while a canvas may excel at overview. The study is therefore designed to reveal task-conditional tradeoffs rather than produce a simplistic winner.

3.4 Measures

We collect both behavioral and subjective measures. Behavioral measures include task completion time, rubric-based task quality, intervention count, time-to-first-useful intervention, and recovery latency during the failure scenario. Because a central theoretical claim concerns mental model formation, we also include a plan-recall measure after each condition: participants reconstruct the workflow structure and agent roles from memory. Subjective measures cover perceived workload, transparency, control, monitoring confidence, and willingness to reuse. We describe workload in NASA-TLX style terms, but in the current package we operationalize the measure as compact mental-demand and effort items to keep the local instrumentation lightweight.

Qualitative data come from think-aloud behavior, post-condition free responses, and a final interview about breakdowns and hybrid feature preferences. The qualitative component is essential because some of the most important differences between representational modalities may not appear in aggregate timing alone. For example, a participant might complete a task quickly in both conditions while still reporting that one condition required substantial mental reconstruction or ad hoc note-taking. We therefore treat qualitative strategy reports not as decoration but as part of the mechanism-level explanation.

3.5 Synthetic dry run

To validate the pipeline before collecting real data, we generated a synthetic dataset that follows the exact schema described above. The dry run instantiates 24 planned participants across two interface conditions and three tasks per condition, yielding 144 trial rows. Two participants are intentionally marked as excluded so that preprocessing logic can be exercised; the retained dataset therefore contains 22 participants and 132 usable trial rows. The generation script parameterizes task- and condition-specific centers for completion time, success, intervention count, workload, transparency, control, recall, and reuse. These centers encode the hypothesized tradeoff: the canvas should support monitoring and recall, whereas the terminal should support direct recovery.

The analysis script then performs the same operations intended for the real study. It excludes invalid participants, computes derived metrics, produces participant-level and condition-level summaries, writes qualitative theme counts, and generates manuscript-ready plots under the local output directory. Importantly, the dry run does not perform inferential statistics. Because the data are synthetic, the correct question is not statistical significance but whether the measurement plan generates interpretable outputs and whether the manuscript can describe them without contradiction.

3.6 Analysis linkage

We designed the analysis plan to align closely with the conceptual claims in the paper. If representation changes planning and monitoring, then the canvas condition should show clearer advantages on monitoring success, transparency, and plan recall. If representation changes repair cost, then the terminal condition should show clearer advantages on recovery time and command precision. If representation changes subjective burden, then differences

Table 1: Condition-level descriptive summary from the synthetic dry run. These numbers validate the analysis structure only.

Condition	Success	Workload	Transparency	Recall	Time (s)
Canvas	82.5	4.11	5.64	4.21	684.8
Terminal	79.3	4.68	5.41	2.99	684.6

in workload and qualitative strategy reports should reveal where users are compensating for hidden state.

The current manuscript therefore treats the dry run as a coherence test for this explanatory chain. The point is not to say “the canvas wins” or “the terminal wins.” The point is to show that a CHI-style comparison can connect theory, instrumentation, results structure, and design implications in a way that is ready for actual deployment.

4 RESULTS

This section reports *synthetic dry-run* outputs only. The numbers below do not come from real participants and should not be interpreted as confirmatory evidence. We report them because they validate the measurement logic, expose how the comparison would read in a paper, and surface whether the hypothesized tradeoff is legible in the planned analysis.

4.1 Interface-level descriptive pattern

At the interface level, the synthetic pattern suggests that the canvas condition may support better overall coordination even when mean completion time remains nearly unchanged. Across all retained trial rows, the canvas condition produced a mean task success score of 82.5% compared with 79.3% for the terminal condition. The canvas condition also produced lower workload (4.11 versus 4.68 on a 7-point scale), slightly higher transparency (5.64 versus 5.41), and substantially higher plan recall (4.21 versus 2.99). Mean completion time was effectively equivalent overall: approximately 685 seconds in both conditions. That equivalence is meaningful. It implies that a spatial interface need not be slower in aggregate even if it introduces interaction overhead on some tasks, because the overhead can be offset by reduced reconstruction effort elsewhere.

The larger gap in recall than in transparency is especially important. A terminal can be transparent in the narrow sense that it shows a faithful chronological record of what happened. Yet that does not guarantee that users can reconstruct the larger plan after the fact. The dry run therefore supports a distinction between *local action transparency* and *global workflow reconstructability*. In a real study, that distinction would likely matter when participants must return to a partially completed workflow after attention has shifted elsewhere.

4.2 Task-specific tradeoffs

Task-level summaries reveal a more differentiated story than the overall means alone. The strongest canvas advantage appears in the monitoring task. There the canvas condition reached 85.2% task success and 4.57 plan-recall points, while the terminal condition

Table 2: Task-by-condition descriptive summary from the synthetic dry run.

Task	Condition	Success	Recall	Time (s)
Delegation	Canvas	83.6	3.94	707.8
Delegation	Terminal	81.5	2.90	612.4
Monitoring	Canvas	85.2	4.57	762.6
Monitoring	Terminal	72.4	2.75	902.1
Recovery	Canvas	78.6	4.11	584.2
Recovery	Terminal	84.0	3.32	539.4

reached 72.4% task success and 2.75 recall points. Mean completion time was also lower in the canvas condition for monitoring (762.6 seconds versus 902.1 seconds). This pattern aligns with the theoretical expectation that spatial layout helps users compare branches, notice stalled dependencies, and preserve a sense of the whole.

Delegation shows a more balanced tradeoff. In the dry run, the terminal condition is faster for delegation (612.4 seconds versus 707.8 seconds), but the canvas condition is slightly more successful (83.6% versus 81.5%) and more memorable (3.94 versus 2.90 recall points). This is a useful reminder that efficient command issuance does not necessarily imply better planning. A terminal can make assigning work fast, but a canvas can make missing structure visible before the work begins.

Recovery flips the pattern. Here the terminal condition is both faster and more successful in the dry run: 539.4 seconds and 84.0% success versus 584.2 seconds and 78.6% success for the canvas. Recovery latency is also lower in the terminal condition (61.8 seconds versus 73.0 seconds). These values match the intended rationale for the terminal condition. Once users know what needs to be fixed, compact commands such as pause, inspect, rebind, and rerun can make repair operations more direct than drag-and-reconnect interactions on a board. The implication is not that spatial interfaces are poor for recovery, but that their benefits may lie more in upstream diagnosis than in the final surgical action.

These task-level contrasts are valuable because they prevent an overly broad conclusion. If we reported only the overall averages, the paper might imply that the canvas was simply “better” because it improved recall and reduced workload without changing mean time. The task breakdown instead suggests a more realistic design interpretation: spatial representations may help users anticipate and diagnose coordination problems, while terminal representations may help them execute precise repairs once a problem is located.

4.3 Qualitative dry-run themes

The synthetic qualitative notes further sharpen this interpretation. Canvas-side notes consistently emphasized spatial overview, branch visibility, and the ability to notice missing roles or stale dependencies at a glance. In the generated theme counts, the canvas condition strongly expressed a spatial-overview theme across all 66 retained task notes. Participants described using the board layout to see whether any role or dependency was missing, to compare branches without rereading the whole history, and to trace a stale writer branch back to a conflicting edge. The friction described for

the canvas condition was different. Rather than feeling lost globally, users reported wanting keyboard-like shortcuts and faster repair gestures once they had already understood the situation.

Terminal-side notes told the complementary story. Terminal participants repeatedly emphasized fast command repair and the usefulness of a compact chronological record. In the generated theme counts, fast-command repair appeared in all 66 retained terminal notes. At the same time, terminal notes also consistently expressed mental reconstruction costs. Participants reported keeping a mental checklist of missing steps, scrolling to compare branches, and inferring upstream structure from warning lines or inspection commands. This combination is important. It suggests that the terminal condition is not opaque in the sense of hiding individual actions. Instead, it can be cognitively heavy because the user must continuously transform a chronological record into a structural model.

One particularly useful theme in the dry run was hybrid demand. Both conditions generated notes that asked for properties of the other one. Canvas users wanted quick keyboard interventions. Terminal users wanted side-by-side or branch-level overview. The fact that hybrid requests emerged under both conditions reinforces the paper’s claim that this comparison is not about selecting a universal winner. It is about exposing complementary strengths that future systems should combine more deliberately.

4.4 What the dry run validates

Because the dataset is synthetic, the most important result is not a number but a methodological outcome. The dry run shows that the study structure can produce interpretable contrasts at three levels: overall interface pattern, task-specific tradeoff, and qualitative mechanism. It also shows that the analysis scripts, summary tables, and argument structure align. Monitoring-heavy work generates the kind of canvas advantage the theory predicts. Recovery-heavy work generates the kind of terminal advantage that terminal-side design arguments would predict. And the qualitative notes explain why those contrasts arise in terms of overview, memory offloading, and precise intervention.

That coherence matters for future empirical work. A real study should not adopt the same numbers, but it can adopt the same explanatory logic: if representation changes what users must reconstruct, then differences should appear most clearly in monitoring and recall; if representation changes repair friction, then differences should appear most clearly in recovery time and intervention style. In this sense, the synthetic results function as a rehearsal for the research question rather than an answer to it.

5 DISCUSSION

The dry run suggests that the most productive way to compare canvas and terminal interfaces is not through a binary question of superiority, but through a representational account of when each interface externalizes or compresses the right kind of work. This account has implications for both HCI research and system design.

First, the comparison highlights that multi-agent supervision is a form of distributed cognition. Users are not only reading outputs; they are coordinating responsibility, uncertainty, dependency, and repair across several partially autonomous processes. A spatial canvas appears promising when the user’s main challenge is to hold

that distribution in view. It supports the kind of external memory that earlier sensemaking work identified as central to complex analytic activity [1]. In our dry run, this appeared through better monitoring performance and stronger plan recall. The likely mechanism is not that a canvas is intrinsically clearer, but that it lets users encode structure into place: who owns what, what depends on what, and where the next intervention should go.

Second, the terminal remains compelling because it compresses action. When users already know the problem and need to fix it precisely, textual commands can minimize the interaction distance between diagnosis and repair. This helps explain why terminal tools continue to matter even as new spatial interfaces are introduced [3]. The dry run's recovery pattern does not undermine the canvas condition; rather, it clarifies the task boundary at which command-line precision may outperform spatial manipulation. A future CHI study should therefore avoid asking whether a canvas "beats" a terminal in the abstract. The more interesting question is how users transition between overview-seeking and repair-seeking states, and how interfaces can support both without forcing users to rebuild context.

Third, our results point toward hybrid interface design. Several concrete implications follow. Future agent interfaces should preserve a persistent plan representation even in command-first environments, so that users do not need to infer workflow structure solely from chronological logs. Conversely, spatial environments should offer low-latency textual intervention for operations such as pause, reset, rebind, and rerun. Branch lineage should be visible at a glance, but it should also be scriptable. Monitoring views should foreground blocked dependencies and stale outputs, while repair views should foreground the exact operation needed to restore correctness. In short, spatial overview and procedural directness should not be treated as mutually exclusive design commitments.

Fourth, the comparison has methodological implications for AI-interface research. Many current systems change model capability, task support, and representation all at once, which makes evaluation difficult to interpret. Our package argues for holding backend behavior constant and varying representation more deliberately. This approach is useful not only for canvas-versus-terminal comparisons, but also for other modality questions such as notebook versus chat, dashboard versus conversational panel, or timeline versus board. If CHI aims to understand how interface form shapes human-AI work, then it needs more matched-comparison studies rather than only system-specific case reports.

Fifth, the paper suggests that representational fit is task-relative and expertise-relative. The terminal may remain preferable for expert users engaged in fast, iterative correction, especially when the workflow already exists mentally or externally elsewhere. The canvas may remain preferable when users must coordinate many branches, notice hidden omissions, or return to work after interruptions. A real study should therefore examine not only average performance but also switching behavior, self-generated workarounds, and the conditions under which participants seek hybrid functionality.

Finally, the dry run demonstrates the value of local-first research packaging. Even before a real study is run, a repository can embody the comparison in artifacts that are inspectable, reproducible,

and extensible. The prototype, scripts, summaries, and manuscript draft make the argument falsifiable. Another researcher could modify the task mix, change the event schema, or replace the synthetic generator with real exports while preserving the same analytic scaffold. In a field where many agent interfaces are released as demos or products without transparent evaluation pathways, that reproducibility is itself a useful HCI contribution.

Our broader claim is therefore modest but important. Representational modality is not a surface-level choice around a fixed intelligence. It shapes when users notice trouble, how they reconstruct state, and whether they feel able to steer long-running agentic work. That claim deserves direct empirical study, and this paper provides a concrete framework for running one.

6 LIMITATIONS

This work has several limitations that substantially constrain interpretation. The most obvious limitation is that the current dataset is synthetic. The descriptive patterns reported here were generated to validate an analysis pipeline and manuscript structure, not to establish empirical truth. They encode hypothesized differences between interface modalities and therefore cannot serve as confirmatory evidence for those differences. A second limitation is that the prototype is a high-fidelity mockup rather than a production system with live models, real latency, or unpredictable agent behavior. This is useful for control, but it may understate the volatility and interruption costs present in real agentic work. A third limitation is task scope. We focus on delegation, monitoring, and recovery because those tasks expose important coordination demands, but real-world workflows may include negotiation, collaborative authoring, or long-horizon planning that change the balance between spatial overview and terminal precision. A fourth limitation is population scope. The intended study targets AI-literate knowledge workers, which means the findings may not transfer directly to novices or to highly specialized expert operators. These limitations are not incidental; they define the current paper as a study-ready framework and synthetic rehearsal rather than a completed empirical comparison.

7 ETHICS AND PRIVACY

The current repository contains no real participant data, which substantially reduces immediate privacy risk. Even so, the design of the future study raises important ethical questions. Multi-agent systems often operate on text that may contain confidential ideas, code, or work artifacts, and participants may inadvertently reveal strategy or workflow information that they did not intend to share. For that reason, early sessions should rely on synthetic tasks rather than participants' live materials. Any future data collection should separate participant identity from exported logs, store only the minimum interaction traces needed for analysis, and explain clearly which actions are recorded, how long those records are retained, and who can inspect them. If screen recordings are used, they should be optional and justified separately because they can capture more context than event logs alone. Researchers also have a responsibility not to overstate what interface results imply about the underlying models. An interface can change users' trust, control, and

sense of visibility without making the agent more reliable. Ethical reporting therefore requires careful distinction between interaction quality and model competence, especially in domains where hidden failures could lead users to delegate more authority than the system deserves.

8 CONCLUSION

The comparison between infinite-canvas and terminal interfaces is rapidly becoming practical rather than hypothetical in multi-agent systems, yet HCI still lacks a clean way to study it. This paper contributes a matched comparison framework, a local-first prototype, a study design, and a synthetic dry run that together make the problem concrete. The dry run suggests a plausible complementarity: canvases help users see and remember workflow structure, while terminals help users execute direct repairs. Because the current evidence is synthetic, these patterns remain hypotheses rather than findings. Even so, the package shows how future CHI work can treat agent-interface representation as a primary variable and can design hybrid systems that combine spatial overview with command-level precision.

REFERENCES

- [1] Christopher Andrews, Alex Endert, and Chris North. 2010. Space to Think: Large, High-Resolution Displays for Sensemaking. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, 55–64. <https://doi.org/10.1145/1753326.1753336>
- [2] Ian Arawjo, Chelse Swoopes, Priyan Vaithilingam, Martin Wattenberg, and Elena L. Glassman. 2024. ChainForge: A Visual Toolkit for Prompt Engineering and LLM Hypothesis Testing. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, Article 523, 18 pages. <https://doi.org/10.1145/3613904.3642016>
- [3] Alexandre De Masi. 2026. Terminal Is All You Need: Design Properties for Human-AI Agent Collaboration. *arXiv preprint arXiv:2603.10664* (2026). <https://doi.org/10.48550/arXiv.2603.10664> Accepted as poster at the CHI 2026 Workshop on Human-AI-UI Interactions Across Modalities.
- [4] Victor Dibia, Jingya Chen, Gagan Bansal, Suff Syed, Adam Fourney, Erkang Zhu, Chi Wang, and Saleema Amershi. 2024. AUTOGEN STUDIO: A No-Code Developer Tool for Building and Debugging Multi-Agent Systems. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, Miami, Florida, USA, 72–79. <https://doi.org/10.18653/v1/2024.emnlp-demo.8>
- [5] Will Epperson, Gagan Bansal, Victor Dibia, Adam Fourney, Jack Gerrits, Erkang Zhu, and Saleema Amershi. 2025. Interactive Debugging and Steering of Multi-Agent AI Systems. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3706598.3713581>
- [6] K. J. Kevin Feng, Kevin Pu, Matt Latzke, Tal August, Pao Siangliulue, Jonathan Bragg, Daniel S. Weld, Amy X. Zhang, and Joseph Chee Chang. 2026. Cocoa: Co-Planning and Co-Execution with AI Agents. *arXiv preprint arXiv:2412.10999* (2026). <https://doi.org/10.48550/arXiv.2412.10999> CHI 2026 paper.
- [7] Minsuk Kahng, Ian Tenney, Mahima Pushkarna, Michael Xieyang Liu, James Wexler, Emily Reif, Krystal Kallarackal, Minsuk Chang, Michael Terry, and Lucas Dixon. 2025. LLM Comparator: Interactive Analysis of Side-by-Side Evaluation of Large Language Models. *IEEE Transactions on Visualization and Computer Graphics* 31, 1 (2025), 503–513. <https://doi.org/10.1109/TVCG.2024.3456354>
- [8] Tae Soo Kim, Yoonjoo Lee, Jamin Shin, Young-Ho Kim, and Juho Kim. 2024. EvalLM: Interactive Evaluation of Large Language Model Prompts on User-Defined Criteria. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, Article 306, 21 pages. <https://doi.org/10.1145/3613904.3642216>
- [9] Yanna Lin, Leni Yang, Haotian Li, Huamin Qu, and Dominik Moritz. 2025. InterLink: Linking Text with Code and Output in Computational Notebooks. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*. <https://doi.org/10.48550/arXiv.2502.16114> Accepted at CHI 2025.
- [10] Aditi Mishra, Utkarsh Soni, Anjana Arunkumar, Jinbin Huang, Bum Chul Kwon, and Chris Bryan. 2025. PromptAid: Prompt Exploration, Perturbation, Testing and Iteration using Visual Analytics for Large Language Models. *arXiv preprint arXiv:2304.01964* (2025). <https://doi.org/10.48550/arXiv.2304.01964>
- [11] Hussein Mozannar, Gagan Bansal, Cheng Tan, Adam Fourney, Victor Dibia, Jingya Chen, Jack Gerrits, Tyler Payne, Matheus Kunzler Maldaner, Madeleine Grunde-McLaughlin, Eric Zhu, Griffin Bassman, Jacob Alber, Peter Chang, Ricky Loynd, Friederike Niedtner, Ece Kamar, Maya Murad, Rafah Hosn, and Saleema Amershi. 2025. Magentic-UI: Towards Human-in-the-loop Agentic Systems. *arXiv preprint arXiv:2507.22358* (2025). <https://doi.org/10.48550/arXiv.2507.22358>
- [12] Tongshuang Wu, Ellen Jiang, Aaron Donsbach, Jeff Gray, Alejandra Molina, Michael Terry, and Carrie J. Cai. 2022. PromptChainer: Chaining Large Language Model Prompts through Visual Programming. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts*. Association for Computing Machinery, New York, NY, USA, 1–10. <https://doi.org/10.1145/3491101.3519729> Also available as arXiv:2203.06566.